# Implementing LIME from Scratch with GPU Acceleration

*Milestone 1*

---

Parallel and Distributed Computing

**Group 9**


| | |
|---|---|
| Haris Khalid | 28557 |
| Aadesh Panjwani | 29303 |
| Hamna Hammad | 29003 |
| Bareerah Shahid | 29385 |

# 0 Contents

# 1 Project Overview

LIME (Local Interpretable Model-agnostic Explanations) is a widely used algorithm in explainable AI that explains individual predictions of any black-box classifier by fitting a simple, interpretable surrogate model locally around the prediction. For a single input instance, the algorithm generates a large set of perturbed neighbor samples, runs inference on them via the black-box model, computes proximity-based weights, and then trains a weighted linear regression to extract feature importance.

The core computational challenge is that LIME is designed to explain *one prediction at a time*, and doing so requires generating thousands of perturbed samples, running forward-pass inference on all of them, and computing pairwise distances. This scales poorly on a CPU when either the number of samples or the feature dimensionality is large, making it a natural candidate for GPU acceleration.

This project implements LIME from scratch in three milestones, progressively replacing CPU components with custom GPU kernels. This report covers the work completed so far, the current state of the codebase, and the plan for the remaining milestones.

## 1.1 LIME Pipeline

At a high level, LIME performs the following steps for a given input instance $x$:

1. **Perturbation:** Generate $N$ perturbed samples $\{x_i\}_{i=1}^{N}$ around $x$ using feature masking or noise injection.
2. **Inference:** Run the black-box model $f$ on all perturbed samples to get predictions $\hat{y}_i = f(x_i)$.
3. **Weight Computation:** Compute a proximity weight $w_i$ for each sample based on its distance from the original input $x$.
4. **Surrogate Training:** Fit a weighted linear regression on $(x_i, \hat{y}_i, w_i)$ to obtain feature attributions (coefficients).
5. **Explanation:** The surrogate model's coefficients represent each feature's contribution to the prediction.

Steps 1–3 are the computationally intensive parts and form the focus of our first milestone.

## 2 Division of Tasks

The group consists of four members. Responsibilities are divided to ensure modular development, independent correctness verification, and clean integration across milestones.

### 2.1 Milestone 1

- **Haris Khalid**: Designed the CUDA kernels for feature masking and noise injection, implemented thread-local PRNG-based mask generation on GPU, and validated GPU perturbations against a CPU NumPy reference.
- **Aadesh Panjwani**: Implemented the binary logistic regression classifier, developed the forward-pass inference using cuBLAS-backed matrix operations, optimised memory access patterns, and validated GPU predictions against CPU results.
- **Hamna Hammad**: Implemented both Euclidean and cosine distance kernels using shared-memory parallel reduction, implemented the exponential kernel weighting function, and ensured numerical stability with appropriate clamping.
- **Bareerah Shahid** : Handled host-side orchestration and device memory transfers, integrated all three GPU modules into the end-to-end pipeline, performed CPU vs. GPU performance measurements across a grid of sample and feature sizes, and documented kernel design and data flow.

Milestone 1 implements the GPU-accelerated computational core of LIME: perturbation generation, model inference, and similarity-based weight computation. No deep learning frameworks are used; all GPU work is done through **CuPy RawKernels** (CUDA).

# 3 Project Structure

## 3.1 Repository

The submission is organised as four Python files. In the notebook, each file is written out via `%%writefile`; in the final repository they exist as standalone modules:

| File | Responsibility |
|---|---|
| `lime_common.py` | Shared dataclasses, constants, and utility functions |
| `perturbation.py` | CUDA kernels for binary-mask generation and application |
| `inference.py` | GPU-based logistic regression forward pass |
| `weights.py` | CUDA kernels for Euclidean/cosine distance and kernel weighting |
| `main.py` | End-to-end driver: validates and benchmarks all three modules |

## 3.2 Module 1: Perturbation Generation (`perturbation.py`)

### 3.2.1 What it does

For a given input vector $x \in \mathbb{R}^d$ and a desired sample count $N$, this module generates $N$ perturbed variants of $x$ using **feature masking**. Each perturbation is produced by independently zeroing out each feature with probability $(1 - p_{\text{keep}})$ where $p_{\text{keep}} = 0.5$ by default. Concretely:

$$m_{s,f} \sim \text{Bernoulli}(p_{\text{keep}}), \quad s \in [N],\ f \in [d] \tag{1}$$

$$\tilde{x}_{s,f} = x_f \cdot m_{s,f} \tag{2}$$

### 3.2.2 CUDA Design

Two kernels are used:

**Kernel 1: `generate_mask_kernel`** This kernel assigns one thread per element of the $N \times d$ mask matrix (total $N \cdot d$ threads). Each thread runs an independent Xorshift32 PRNG seeded with the combination of a global seed and the thread index. The use of a thread-local PRNG means there are no atomic operations or shared-memory conflicts, making the generation embarrassingly parallel.

The Xorshift32 update per thread is:

Listing 1: Xorshift32 PRNG in the mask kernel

```
unsigned int state = (unsigned int)(seed ^ (idx * 0x27d4eb2d));
if (state == 0) state = 1;
state ^= state << 13;
state ^= state >> 17;
state ^= state << 5;
float r = (float)(state & 0x00FFFFFF) / (float)0x01000000;
d_masks[idx] = (r < keep_prob) ? 1 : 0;
```

**Kernel 2:** `apply_mask_kernel` This kernel uses a 2-D grid: the $x$-dimension covers features and the $y$-dimension covers samples. Each thread computes one element $\tilde{x}_{s,f} = x_f \cdot m_{s,f}$. The 2-D layout makes the index arithmetic straightforward and avoids any inter-thread communication.

#### 3.2.3 CPU Reference and Validation

A CPU reference implementation uses NumPy's `default_rng` with an independent seed and verifies two properties: (1) the empirical keep rate is within a small statistical margin of 0.5, and (2) the element-wise error between expected and actual masked samples is below $10^{-5}$.

### 3.3 Module 2: Logistic Regression Inference (`inference.py`)

#### 3.3.1 What it does

The "black-box" model used for this milestone is a binary logistic regression with weights $w \in \mathbb{R}^d$ and bias $b$. Given the perturbed sample matrix $\tilde{X} \in \mathbb{R}^{N \times d}$, inference computes:

$$\hat{y}_s = \sigma(\tilde{X}_{s,:} \cdot w + b) = \frac{1}{1 + e^{-(\tilde{X}_{s,:} \cdot w + b)}} \tag{3}$$

#### 3.3.2 GPU Implementation

Inference is implemented in two CuPy operations:

Listing 2: GPU inference using cuBLAS via CuPy

```
logits = buf.d_samples @ model.d_weights + model.bias
preds  = (1.0 / (1.0 + cp.exp(-logits))).astype(cp.float32)
```

The matrix-vector multiplication is dispatched to cuBLAS internally by CuPy, which is the standard and most performant approach for this operation. This is a deliberate choice: writing a custom GEMV kernel would not outperform cuBLAS since that library is already highly optimised for this exact pattern.

Note that for small configurations (e.g., 512 samples, 32 features), the GPU inference can be *slower* than the CPU reference. This is expected and not a defect. At small problem sizes the CUDA kernel launch overhead and PCIe data movement dominate the compute time. The GPU advantage emerges as $N$ and $d$ grow, which the benchmark sweep demonstrates clearly (Section 4).

### 3.4 Module 3: Distance and Kernel Weights (`weights.py`)

#### 3.4.1 What it does

For each perturbed sample $\tilde{x}_s$, this module computes a weight $w_s$ reflecting how close it is to the original input $x$. The weight uses an exponential (RBF) kernel over a chosen distance metric:

$$w_s = \exp\left(-\frac{d(x,\, \tilde{x}_s)^2}{2\sigma^2}\right) \tag{4}$$

Two distance metrics are supported:

**Euclidean distance**

$$d_{\text{euc}}(x, \tilde{x}_s) = \|x - \tilde{x}_s\|_2 = \sqrt{\sum_{f=1}^{d}(x_f - \tilde{x}_{s,f})^2} \tag{5}$$

**Cosine distance**

$$d_{\cos}(x, \tilde{x}_s) = 1 - \frac{x \cdot \tilde{x}_s}{\|x\|_2 \|\tilde{x}_s\|_2} \tag{6}$$

The kernel bandwidth $\sigma$ is set adaptively:

$$\sigma = \begin{cases} 0.75\sqrt{d} & \text{Euclidean} \\ 0.25 & \text{Cosine} \end{cases} \tag{7}$$

#### 3.4.2 CUDA Design

**Distance kernels (one block per sample, shared-memory parallel reduction)** Both the Euclidean and cosine kernels follow the same structure: each CUDA block is assigned to one sample $s$. Threads in the block stride over the feature dimension in chunks of `blockDim.x`, accumulate partial results into shared memory, and then a tree reduction collapses the partial sums down to a single value written by thread 0.

For Euclidean distance, shared memory holds one float array of size `blockDim.x`. For cosine distance, three arrays of the same size are needed (dot product, $\|x\|^2$, $\|\tilde{x}_s\|^2$) and all three reductions happen simultaneously, which keeps the cosine kernel efficient without launching extra passes.

The number of threads per block is rounded up to the nearest power of two (capped at 256) so the reduction tree is valid.

**Weight kernel:** After distances are computed, the exponential kernel weight is a per-element operation with no data dependencies between samples, so it maps one thread per sample with a standard 1-D grid.

### 3.4.3 Numerical Stability

The cosine distance kernel includes an explicit numerical clamp:

Listing 3: before taking 1 - cos]Cosine similarity clamped to [-1, 1] before taking 1 - cos

```
float cos_sim = (denom > 1e-8f) ? sh_dot[0] / denom : 0.0f;
cos_sim = fminf(1.0f, fmaxf(-1.0f, cos_sim));
d_distances[s] = 1.0f - cos_sim;
```

This prevents floating-point rounding from producing distances slightly outside $[0, 2]$.

# 4 Results and Validation

All experiments ran on a **Tesla T4** (40 SMs, 15.6 GB global memory, CUDA 12) on Google Colab.

## 4.1 Correctness

For all three modules, GPU outputs were compared element-wise against CPU reference implementations. The table below summarises validation results at the default configuration (2048 samples, 64 features, Euclidean metric):

Table 1: Correctness validation, 2048 samples, 64 features, Euclidean

| Module | Metric | Value | Status |
|---|---|---|---|
| Perturbation | Keep rate | 0.4998 | **PASS** ✓(expected $\approx 0.5$) |
| Perturbation | Mask apply error | $< 10^{-5}$ | **PASS** ✓ |
| Inference | Max abs error | $8.94 \times 10^{-8}$ | **PASS** ✓ |
| Distance (Euc.) | Max abs error | $4.77 \times 10^{-7}$ | **PASS** ✓ |
| Kernel weights | Max abs error | $1.79 \times 10^{-7}$ | **PASS** ✓ |

The same validation was run with cosine distance at 4096 samples and 128 features with equally clean results (distances: $1.79 \times 10^{-7}$, weights: $4.77 \times 10^{-7}$).

## 4.2 End-to-end Pipeline Output (Default Run)

Running `python main.py` produces:

Listing 4: main.py output, 2048 samples, 64 features, Euclidean

```
Device name:          Tesla T4
Total memory:         15.6 GB   |   SM count: 40

MODULE 1, Perturbation Generation
  GPU perturbation time:      0.241 ms
  CPU perturbation time:      1.298 ms
  Speedup (perturbation):     5.39x
  Mask keep rate:             0.4998  (expected ~0.5000)
  Mask application:           PASS \checkmark

MODULE 2, Logistic Regression Inference
  Method:   cuBLAS matmul (d_samples @ d_weights + bias)
  GPU inference time:         0.361 ms
  CPU inference time:         0.259 ms
  Speedup (inference):        0.72x
  Max abs error:              8.94e-08   PASS

MODULE 3, Distance & Kernel Weights
  Sigma:  6.0000  (euclidean: 0.75 * sqrt(64))
  GPU weights time:           0.303 ms
  CPU weights time:           0.778 ms
```

```
  Speedup (weights):          2.56x
  Max abs error (distances):  4.77e-07   PASS \checkmark
  Max abs error (weights):    1.79e-07   PASS \checkmark

PIPELINE SUMMARY
  GPU total (pert + inf + wt):   0.905 ms
  CPU total:                     2.335 ms
  Overall GPU speedup:           2.58x
```

### 4.3 Benchmark Sweep

Running `python main.py --sweep` executes the full pipeline across a grid of sample and feature counts. The results illustrate how GPU speedup scales with problem size:

Table 2: End-to-end pipeline benchmark sweep (Tesla T4, Euclidean metric)

| **Samples** | **Features** | **GPU (ms)** | **CPU (ms)** | **Speedup** |
|---|---|---|---|---|
| 512 | 32 | 0.56 | 0.39 | 0.70× |
| 512 | 64 | 0.60 | 0.54 | 0.91× |
| 512 | 128 | 0.67 | 1.01 | 1.49× |
| 512 | 256 | 0.70 | 1.68 | 2.40× |
| 1024 | 32 | 0.40 | 0.33 | 0.82× |
| 1024 | 64 | 0.42 | 0.57 | 1.36× |
| 1024 | 128 | 0.36 | 1.10 | 3.01× |
| 1024 | 256 | 0.43 | 2.07 | 4.76× |
| 2048 | 32 | 0.43 | 0.59 | 1.37× |
| 2048 | 64 | 0.37 | 1.02 | 2.73× |
| 2048 | 128 | 0.42 | 2.10 | 5.06× |
| 2048 | 256 | 0.74 | 10.57 | **14.31**× |
| 4096 | 32 | 1.68 | 3.73 | 2.22× |
| 4096 | 64 | 1.89 | 5.28 | 2.80× |
| 4096 | 128 | 1.57 | 14.95 | 9.50× |
| 4096 | 256 | 1.32 | 26.97 | **20.48**× |

The GPU is slower than the CPU at very small sizes (512 samples, 32 features) due to kernel launch and data transfer overhead, but achieves up to **20.48× speedup** at 4096 samples and 256 features. The crossover point occurs roughly around 512 samples with 128+ features.

The cosine metric run at 4096 samples, 128 features gave an overall pipeline speedup of **6.96**×.

# 5 How to Run the Code

## 5.1 Prerequisites

- Python 3.9 or later
- A CUDA-capable NVIDIA GPU (CUDA 11.x or 12.x)
- The following Python packages:

```
pip install cupy-cuda12x numpy   # For CUDA 12.x
# OR
pip install cupy-cuda11x numpy   # For CUDA 11.x
```

On Google Colab (which uses CUDA 12), `cupy-cuda12x` is the correct package and is often already installed. `numpy` is always pre-installed on Colab.

## 5.2 Repository Layout

After cloning, the relevant files for Milestone 1 are:

```
project/
  lime_common.py      # Dataclasses and constants
  perturbation.py     # CUDA perturbation kernels
  inference.py        # GPU inference
  weights.py          # CUDA distance and weight kernels
  main.py             # Driver script
```

## 5.3 Running the Default Pipeline

The default run uses 2048 samples, 64 features, and Euclidean distance:

```
python main.py
```

This will print GPU device information, per-module benchmarks and validation results, and a summary table of sample predictions with their distances and weights.

## 5.4 Command-Line Options

`main.py` accepts several arguments:

```
python main.py --samples 4096 --features 128 --metric cosine
```

| Argument | Default | Description |
|---|---|---|
| `--samples` | 2048 | Number of perturbed samples to generate |
| `--features` | 64 | Feature dimensionality |
| `--metric` | euclidean | Distance metric: `euclidean` or `cosine` |
| `--seed` | 42 | Random seed for reproducibility |
| `--sweep` | (flag) | Run a full benchmark sweep over sample/feature grid |

### 5.5 Running the Benchmark Sweep

```
python main.py --sweep
```

This iterates over combinations of {512, 1024, 2048, 4096} samples and {32, 64, 128, 256} features, printing a performance table.

### 5.6 Running on Google Colab

If running from the Jupyter notebook submission:

1. Open the notebook in Colab and set the runtime to **T4 GPU** (Runtime → Change runtime type → T4 GPU).
2. Run all cells sequentially. The `%%writefile` cells write the `.py` files to disk; subsequent `!python main.py` cells execute them.
3. No additional setup is required; all dependencies are pre-installed on Colab's GPU runtime.

### 5.7 Importing Modules Individually

Each module can be imported independently for testing:

Listing 5: Using modules individually

```
import numpy as np
from perturbation import perturbation_generate,
    validate_perturbation
from inference    import model_create, inference_run,
    validate_inference
from weights      import weights_compute, validate_weights

h_original = np.random.rand(64).astype(np.float32)
h_weights  = np.random.rand(64).astype(np.float32) - 0.5

buf   = perturbation_generate(h_original, n_samples=2048,
    n_features=64)
model = model_create(h_weights, bias=0.1)
wb    = inference_run(buf, model)
weights_compute(wb, buf, h_original, metric="euclidean")

print(validate_perturbation(buf, h_original))
print(validate_inference(wb, buf, h_weights, bias=0.1))
print(validate_weights(wb, buf, h_original))
```

# 6 Design Decisions and Discussion

## 6.1 Why CuPy RawKernels instead of pure CUDA C?

The project requirements specify explicit GPU programming via CUDA or OpenCL. CuPy's `RawModule` allows embedding raw CUDA C kernel code as strings inside Python, compiling them at runtime with NVRTC, and launching them with precise grid/block configurations. This gives full control over kernel design (shared memory, thread layout, reduction patterns) while keeping the host code clean and runnable in a standard Python environment without a separate CUDA compilation step. It satisfies the spirit of the requirement: the actual parallel computation is written in and executed as CUDA C.

## 6.2 Xorshift32 PRNG vs. cuRAND

Using a simple thread-local Xorshift32 instead of cuRAND was an explicit choice. cuRAND requires a state array proportional to the number of threads, which adds device memory overhead and an initialisation step. Xorshift32 is seeded per-thread from the global seed and thread index, requires no state array, and produces sufficiently uniform mask distributions for LIME's purposes (the statistical quality of the mask distribution is not a bottleneck for explainability quality). The validation output confirms the keep rate stays within 0.0002 of the target 0.5.

## 6.3 Inference via cuBLAS rather than a Custom Kernel

A custom GEMV kernel was not written for inference, and this was intentional. The matrix-vector product $\tilde{X}w$ is exactly the pattern that cuBLAS is optimised for, and any hand-written kernel would perform significantly worse than cuBLAS unless it implemented tiling and register blocking to the same level of sophistication. Using CuPy's `@` operator dispatches to cuBLAS and is the correct engineering choice. A custom kernel becomes worthwhile when the operation has non-standard access patterns or fused computations that cuBLAS cannot express.

## 6.4 Kernel Warmup

All modules include a warmup step (`_warmup()`) that runs a small dummy computation before benchmarking begins. CUDA's JIT compiler (NVRTC) compiles kernels on first use, which can add tens to hundreds of milliseconds of one-time overhead. Without warmup, the first timing measurement is contaminated by compile time and gives a misleadingly slow result. Warmup ensures benchmark numbers reflect steady-state GPU performance.

## 6.5 GPU Slower Than CPU at Small Sizes

At small problem sizes (e.g., 512 samples, 32 features), the CPU is faster than the GPU. This is not a bug. The overhead of CUDA kernel launch, device synchronisation, and data transfer can easily exceed the actual compute time when the problem is small. For production LIME, this would typically be handled by batching multiple explanations together. For this project the benchmark sweep documents this threshold clearly.

# 7 Remaining Work

## 7.1 Milestone 2: Framework-based Surrogate Training

Milestone 2 completes the full LIME pipeline by training a surrogate model using a high-level ML framework (PyTorch or scikit-learn). It takes the outputs of Milestone 1, perturbed samples, model predictions, and kernel weights, as input, and fits a weighted linear regression to extract feature attributions. The primary goal of this milestone is correctness and interpretability; it serves as the reference implementation before the low-level GPU surrogate is built in Milestone 3.

## 7.2 Milestone 3: Fully GPU-Native Surrogate Training

Milestone 3 replaces the framework-based surrogate from Milestone 2 with a custom CUDA implementation, resulting in a fully GPU-native LIME pipeline where all five steps execute on the GPU with minimal host-device data movement. The surrogate training will be implemented using the closed-form normal equation and, if numerically necessary, an iterative gradient descent alternative. A three-way performance comparison, CPU-only, M1+M2 hybrid, and fully GPU-native, will form the core of the final evaluation.

# 8 Conclusion

Milestone 1 delivers a correct, modular, and benchmarked GPU implementation of the three most computationally expensive stages of the LIME pipeline. All three modules pass element-wise validation against CPU references with errors comfortably below $10^{-4}$. At the largest tested configuration (4096 samples, 256 features), the GPU pipeline completes in 1.32 ms compared to 26.97 ms on the CPU, giving a speedup of **20.48**×.